

● STATESET

WHITE PAPER

MARCH 2026

V0.8.0

STATESET ICOMMERCE PLATFORM

The Embedded Commerce Engine for the *Agentic Era*

A 242,000-line Rust platform that gives every application — from AI agents to enterprise ERPs — a complete commerce stack with zero external dependencies. The SQLite of commerce.

242K

LINES OF RUST

4,249

PASSING TESTS

21

CRATES

0

CLIPPY WARNINGS

10

LANGUAGE BINDINGS

I — EXECUTIVE SUMMARY

Commerce infrastructure should be a library, not a service.

StateSet iCommerce is a Rust-native, embeddable commerce engine designed for a world where AI agents autonomously negotiate, purchase, and settle transactions on behalf of humans and organizations. It delivers the full operational surface of enterprise commerce — orders, inventory, payments, fulfillment, accounting, subscriptions, and 40+ additional domains — as a single library call.

Built on Edition 2024 Rust with `deny(unsafe_code)` enforced across every production crate, iCommerce achieves the reliability profile of database engines while maintaining the ergonomics of a modern SDK. Its dual-backend architecture — embedded SQLite for zero-configuration deployments, PostgreSQL for production scale — means the same code runs in a CLI tool, a WASM module, a mobile app, or a cloud-native microservice.

The platform introduces the Agent-to-Agent (A2A) commerce protocol, enabling autonomous agents to discover capabilities, negotiate quotes, execute escrow-backed purchases, and resolve disputes — all with cryptographic verifiability through the VES v1.0 protocol.

"We didn't set out to build another commerce platform. We set out to make commerce disappear — to make it as invisible and reliable as the filesystem."

DESIGN PRINCIPLE

II — THE PROBLEM

Commerce is still a monolith problem.

Three structural failures define today's commerce infrastructure. Each one compounds in the agentic era, where machines — not humans — are the primary actors in the transaction graph.

1. The SaaS Tax

Modern commerce platforms charge per-transaction fees on infrastructure that should be a commodity. A \$50 order touching Shopify, Stripe, a tax service, and a shipping API accumulates 8–12% in platform costs before the merchant earns a dollar. For AI agents executing thousands of micro-transactions per hour, this fee structure is catastrophic.

2. The Integration Ceiling

Enterprise commerce requires stitching together 15–25 services: an OMS, a WMS, an ERP's AP/AR/GL modules, tax engines, payment processors, subscription billing, fraud detection, and returns management. Each integration is a surface for failure, latency, and data inconsistency. The median enterprise spends 40% of its engineering budget maintaining commerce integrations, not building product.

3. The Agent Gap

AI agents are becoming economic actors — procuring cloud resources, negotiating SaaS contracts, managing supply chains. Yet no commerce infrastructure exists for agent-to-agent transactions. Agents cannot discover each other's capabilities, negotiate programmatic quotes, hold funds in escrow pending delivery verification, or resolve disputes through algorithmic arbitration. Every agent-initiated purchase today requires a human in the loop.

"By 2028, autonomous agents will initiate more commercial transactions than humans. The infrastructure doesn't exist yet."

THESIS



III — THE ARCHITECTURE

21 crates. Zero runtime dependencies.

iCommerce is structured as a Rust workspace of 21 crates, each with a single responsibility. The layered architecture ensures that pure domain logic never touches I/O, database code never knows about HTTP, and the embedded facade composes everything into a single `Commerce::new("store.db")` call.

SDK · FFI · HTTP · CLI · WASM	Consumer Layer	
stateset-embedded	Commerce facade, event bus	
stateset-a2a	Agent-to-Agent protocol	
stateset-http	Axum REST + SSE + OpenAPI	
stateset-db	SQLite PostgreSQL backends	
stateset-sync	Outbox, conflict resolution	
stateset-jobs	Cron, retry, backoff	
stateset-migrations	Schema versioning + checksums	
stateset-core	Pure domain models (0 I/O)	
stateset-primitives	Money, CurrencyCode, typed IDs	
stateset-crypto	VES v1.0, Ed25519, AES-GCM	
stateset-policy	Deny-overrides rule engine	
stateset-pricing	Deterministic pricing engine	
stateset-authz	RBAC + rate limiting + audit	
stateset-observability	Metrics, tracing, SLO eval	
stateset-protocol	Wire-format types (WASM-safe)	
stateset-macros	StateSetId, GenerateDto, etc.	

242,428

4,249

LINES OF RUST	TESTS PASSING
0	0
CLIPPY WARNINGS	UNSAFE BLOCKS

Type Safety by Design

Every entity ID is a distinct newtype — you cannot pass an `OrderId` where a `CustomerId` is expected. The `Money` type pairs amounts with currency codes, making cross-currency arithmetic a compile-time error. `#[non_exhaustive]` on 241 enums ensures forward-compatible evolution. 496 `#[must_use]` annotations prevent silent value drops.

Dual Backend Architecture

Feature flags toggle between SQLite (bundled, zero-config, single-file) and PostgreSQL (connection pooling, async, saga coordination). The same domain models and repository traits underpin both — a test written against `Commerce::new(":memory:")` is valid against a 100-node Postgres cluster. Vector search, event sourcing, and saga patterns activate with additional feature flags.

IV — COMMERCE DOMAINS

46 domain models. One library.

iCommerce delivers the operational surface of an enterprise ERP as composable Rust modules. Each domain is a pure data model with validation, state-machine transitions, and repository traits — no I/O, no side effects, fully testable.

DOMAIN	CAPABILITIES	MODELS
Order Management	Create, confirm, process, ship, cancel with state-machine enforcement	Order, OrderItem, OrderFilter
Inventory	Stock tracking, reservations, adjustments, reorder points, lot/serial tracking	InventoryItem, Lot, Serial
Payments	Card, ACH, crypto, stablecoins (USDC, ssUSD), store credit, gift cards	Payment, X402PaymentIntent
Fulfillment	Wave planning, pick/pack/ship, carton management, label generation	Wave, PickTask, PackTask, ShipTask
Accounting	Double-entry GL, accounts payable, accounts receivable, cost accounting	JournalEntry, Invoice, CreditMemo
Subscriptions	Recurring billing, plan management, renewal, cancellation, proration	Subscription, SubscriptionPlan
Returns & Warranties	RMA processing, refund calculation, warranty claims, quality checks	Return, Warranty, QualityCheck

DOMAIN	CAPABILITIES	MODELS
Manufacturing	Bill of Materials, work orders, assembly operations, cost allocation	BOM, WorkOrder, BOMComponent
Tax & Promotions	Multi-jurisdiction tax, compound rates, stackable promotions, loyalty programs	TaxRule, Promotion, LoyaltyProgram
Analytics & Fraud	Customer segmentation, forecasting, fraud scoring, review management	Segment, FraudCheck, Review

"Most commerce platforms give you 60% of what you need and charge you to build the rest. iCommerce ships with the full ERP surface — AP, AR, GL, cost accounting, manufacturing, fulfillment — because agents don't have time to integrate."

ARCHITECTURE DECISION RECORD #001

The A2A Protocol

Agent-to-Agent commerce is the platform's thesis feature: a protocol layer that enables autonomous AI agents to conduct the full lifecycle of a commercial transaction without human intervention.



Discovery & Identity

Agent Cards describe capabilities, pricing, and trust scores. ERC-8004 identity documents provide verifiable credentials. Reputation tiers (Bronze → Diamond) gate access to high-value operations.



Negotiation & Quotes

Structured RFQ protocol with multi-party matching. Agents submit quotes with pricing, SLAs, and conditions. Marketplace module enables automated bid evaluation and counter-offers.



Escrow & Settlement

Conditional escrow with four condition types. Funds are held until delivery verification, quality inspection, or time-lock expiration. x402 protocol settles in USDC or ssUSD stablecoins on-chain.



Splits & Subscriptions

Percentage or fixed-amount multi-party payment splitting. Recurring A2A subscriptions with automatic renewal. Budget management with spending limits and circuit breakers.



Disputes & SLAs

Algorithmic dispute resolution with evidence hashing and deadline tracking. Service Level Agreements with compliance checking. Escalation paths for human review when confidence is low.



Event Streaming

Real-time event bus with wildcard/prefix filtering. HMAC-SHA256 signed webhook delivery with SSRF protection. Server-Sent Events for live transaction monitoring.

543

A2A PROTOCOL TESTS

186

MCP SERVER TOOLS

53

A2A MCP TOOLS

18

AGENT DEFINITIONS



VI — SECURITY & CRYPTOGRAPHY

VES v1.0 — Verifiable Event Streaming

The VES protocol provides cryptographic guarantees for every event in the commerce stream. Domain-separated hashing, Ed25519 signatures, and Merkle inclusion proofs create an auditable, tamper-evident transaction history.

Cryptographic Primitives

Ed25519 — Deterministic ECDSA signatures for agent identity and event signing. Constant-time verification prevents timing attacks.

AES-256-GCM — Authenticated encryption for payload confidentiality. Random nonces per operation with automatic zeroization of key material.

X25519 ECDH — Ephemeral key agreement for multi-recipient encryption. HKDF-SHA256 key derivation with payload-bound AAD.

Merkle Trees — SHA-256 domain-separated leaf and node hashing. Inclusion proofs enable lightweight verification without full state.

Security Posture

`deny(unsafe_code)` is enforced on every production crate (the FFI crate uses `warn` with documented, audited blocks). `deny(clippy::unwrap_used)` prevents panics in production paths. Constant-time comparison via the `subtle` crate protects hash verification from timing side-channels.

The HTTP layer uses constant-time bearer token comparison, explicit CORS allowlists (no wildcards), token-bucket rate limiting, and SSRF validation on all webhook URLs. The policy engine provides deny-overrides semantics with explainable denial reasons for audit compliance.

Domain Separation

Ten domain prefixes (`VES_PAYLOAD_PLAIN_V1` , `VES_EVENTSIG_V1` , `VES_LEAF_V1` , etc.) prevent cross-domain hash confusion attacks. Cross-language test vectors ensure bit-exact compatibility between Rust and JavaScript implementations.

VII — PLATFORM ECOSYSTEM

Every language. Every runtime.

iCommerce compiles to native libraries for 10 languages, a WASM module for browser and edge deployment, and a standalone HTTP server. The same Rust core powers every surface.

Language Bindings

The FFI crate provides a stable C-ABI surface with `#[repr(C)]` types, ABI versioning, thread-local error storage, and panic catching across the FFI boundary. Language-specific bindings wrap this surface with idiomatic APIs.



MCP Server

The Model Context Protocol server exposes 186 tools for AI agent integration, covering every commerce operation. Agents can create orders, adjust inventory, process returns, calculate tax, and execute A2A transactions through a structured tool-calling interface.

CLI

A branded command-line interface with 143 modules provides interactive commerce management. Built with ES modules, `@clack/prompts` for UX, and branded theming for professional deployment. The CLI doubles as an MCP host, enabling multi-agent orchestration from the terminal.

HTTP Server

The `stateset-http` crate wraps the embedded engine in an Axum-based REST API with OpenAPI 3.1 documentation, Server-Sent Events for real-time streaming, cursor-based pagination, and a `ServerBuilder` fluent API.

Middleware Stack

1. Tracing — structured request logging
2. ETag + Cache-Control — FNV-1a hash, path-aware caching
3. Rate Limiting — token-bucket with burst capacity
4. Bearer Auth — constant-time comparison, tenant binding
5. CORS — explicit origin allowlist, no wildcards
6. Request ID — X-Request-ID propagation for distributed tracing

Observability

Lock-free atomic counters for business metrics (orders, payments, inventory adjustments). RED snapshots (Rate/Errors/Duration) per operation with SLO evaluation. Optional OpenTelemetry export via feature flag. Latency histograms with p50/p95/p99 percentile tracking.

VIII — ENGINEERING QUALITY

Built to the standard of database engines.

iCommerce applies the lint discipline of projects like reth, foundry, and alloy. Every crate inherits workspace-wide lint rules. The result is a codebase where the compiler is the first reviewer.

19_{/21} CRATES WITH DENY(UNWRAP_USED)	241 #[NON_EXHAUSTIVE] ENUMS
496 #[MUST_USE] ANNOTATIONS	57 REPOSITORY TRAITS

Error Handling

Zero uses of `anyhow` — every error is a structured `thiserror` type with categorization helpers (`is_retryable`, `is_not_found`, `suggested_status_code`). Compile-time size assertions on error enums prevent accidental bloat. The `ValidationBuilder` provides 15+ composable validators with first-error or collect-all semantics.

Testing Strategy

4,249 tests span unit tests, integration tests, property-based tests (proptest), snapshot tests (insta), and benchmark regression gates (criterion). Cross-language test vectors ensure Rust and JavaScript

implementations produce bit-identical cryptographic outputs. The test suite runs in under 30 seconds on commodity hardware.

Workspace Lints

The workspace enforces `deny(unused_must_use)`, `deny(rust_2018_idioms)`, and 15 pedantic Clippy lints including `missing_const_for_fn`, `use_self`, `derive_partial_eq_without_eq`, and `doc_markdown`. The result: zero warnings across 242K lines and all test targets.

Performance

Release builds use `opt-level = 3`, thin LTO, symbol stripping, and `panic = "abort"`. A dedicated profiling profile preserves debug symbols for flamegraph analysis. Proptest and rand_chacha are compiled at `opt-level = 3` even in dev builds to keep test latency low.



Commerce is infrastructure. Infrastructure should be embedded.

StateSet iCommerce is open source under MIT / Apache-2.0 dual license.
The platform is available today as Rust crates, language bindings, a CLI, an
HTTP server, and a WASM module.

[VIEW ON GITHUB](#) →

STATESET

[github.com/stateset/stateset-icommerce](#)

MIT / Apache-2.0 · v0.8.0 · Edition 2024 · Rust 1.85+

© 2026 StateSet, Inc.

This document may be freely distributed.